

React Native Notes: Topic 5 — Forms & Input

1. Controlled vs. Uncontrolled Components

In React Native, form inputs can be handled in two primary ways: **Controlled** (where state drives display) and **Uncontrolled** (where internal native state drives display).

Controlled Components

What is it?

A controlled component is an input element where the displayed value is directly driven by React component state (value + onChangeText). React state serves as the **single source of truth**.

Why use it?

- Enables instant, real-time input validation (e.g., dynamic password strength meters).
- Allows conditional UI behavior based on user typing (e.g., disabling submission buttons).
- Simplifies state synchronization across multiple UI components.

When to use it?

Used in almost **95%+ of standard React Native form cases**, including authentication screens, multi-field registration forms, search bars, and chat input fields.

How to use it?

Pass a value prop tied to a React useState hook and update that state via the onChangeText callback.

TypeScript

```
import React, { useState } from 'react';  
import { View, TextInput, Text, StyleSheet } from 'react-native';
```

```
const ControlledInputExample = () => {  
  const [name, setName] = useState("");
```

```

const [email, setEmail] = useState("");

return (
  <View style={styles.container}>
    <Text style={styles.label}>Name:</Text>
    <TextInput
      style={styles.input}
      value={name}
      onChangeText={setName}
      placeholder="Enter your name"
    />

    <Text style={styles.label}>Email:</Text>
    <TextInput
      style={styles.input}
      value={email}
      onChangeText={setEmail}
      placeholder="Enter your email"
      keyboardType="email-address"
    />

    <Text style={styles.preview}>
      Preview: {name} ({email})
    </Text>
  </View>
);
};

const styles = StyleSheet.create({
  container: { padding: 20, flex: 1 },
  label: { fontSize: 16, fontWeight: '600', marginTop: 10 },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    marginTop: 5,
  },
  preview: { marginTop: 20, fontSize: 14, color: '#666' },
});

```

```
});
```

```
export default ControlledInputExample;
```

Uncontrolled Components

What is it?

An uncontrolled component lets the underlying mobile platform handle input state natively. The React component reads stored values via React useRef when needed.

Why use it?

- Prevents re-rendering the component on every single keystroke.
- Useful for lightweight, isolated form elements where React state updates during active typing are unneeded.

When to use it?

- Performance-critical forms with large numbers of dynamic inputs.
- Quick one-off fields (e.g., feedback or contact forms) where you only care about reading values on final submit.

How to use it?

Omit the value prop (or pass defaultValue), and store typed text directly into a mutable ref using onChangeText.

TypeScript

```
import React, { useRef } from 'react';
```

```
import { View, TextInput, Button, Text, StyleSheet, Alert } from 'react-native';
```

```
const UncontrolledInputExample = () => {  
  // Storing input values in refs without triggering component re-renders  
  const nameRef = useRef("");  
  const emailRef = useRef("");
```

```
  const handleGetValues = () => {  
    Alert.alert(  
      'Form Values',  
      `Name: ${nameRef.current}\nEmail: ${emailRef.current}`
```

```

    );
  };

  return (
    <View style={styles.container}>
      <Text style={styles.label}>Uncontrolled Inputs</Text>

      <TextInput
        style={styles.input}
        placeholder="Name"
        defaultValue="John"
        onChangeText={(text) => (nameRef.current = text)}
      />

      <TextInput
        style={styles.input}
        placeholder="Email"
        defaultValue="john@example.com"
        keyboardType="email-address"
        onChangeText={(text) => (emailRef.current = text)}
      />

      <Button title="Get Values" onPress={handleGetValues} />
    </View>
  );
};

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20 },
  label: { fontSize: 18, fontWeight: 'bold', marginBottom: 20 },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    marginBottom: 12,
  },
});

```

```
export default UncontrolledInputExample;
```

2. Essential TextInput Props for Validation & UX

React Native's `TextInput` provides specialized props that alter native device keyboards, control text transformation, and handle validation-adjacent behaviors.

Key Props Overview

- **keyboardType**: Configures the software keyboard layout (e.g., "numeric", "email-address", "phone-pad").
- **maxLength**: Prevents users from entering characters beyond a specific limit.
- **autoCorrect**: Toggles platform spell-check and auto-correction (true / false).
- **returnKeyType**: Customizes the label on the native keyboard's action key (e.g., "done", "go", "next", "search", "send").
- **secureTextEntry**: Masks character input for password privacy.
- **autoCapitalize**: Controls character casing behavior ("none", "sentences", "words", "characters").

Real-Life Example: Form Input Props Showcase

TypeScript

```
import React, { useState } from 'react';
import { ScrollView, View, Text, TextInput, StyleSheet } from 'react-native';
```

```
const TextInputPropsDemo = () => {
  const [phone, setPhone] = useState("");
  const [code, setCode] = useState("");

  return (
    <ScrollView style={styles.container}>
      { /* Numeric Only Keypad with Character Limit */ }
      <Text style={styles.label}>PIN Code (Numeric, Max 4 Chars):</Text>
      <TextInput
        style={styles.input}
        value={code}
        onChangeText={setCode}
        keyboardType="numeric"
        maxLength={4}
        secureTextEntry
```

```

        placeholder="1234"
      />

      {/ * Phone Number Input */}
      <Text style={styles.label}>Phone Number:</Text>
      <TextInput
        style={styles.input}
        value={phone}
        onChangeText={setPhone}
        keyboardType="phone-pad"
        maxLength={15}
        placeholder="+1 (555) 000-0000"
      />

      {/ * Search Input with Custom Return Key */}
      <Text style={styles.label}>Search Query:</Text>
      <TextInput
        style={styles.input}
        placeholder="Type and press search..."
        returnKeyType="search"
        autoCorrect={false}
        autoCapitalize="none"
        onSubmitEditing={() => console.log('Searching...')}
      />
    </ScrollView>
  );
};

const styles = StyleSheet.create({
  container: { flex: 1, padding: 16 },
  label: { fontSize: 14, color: '#333', marginTop: 12, marginBottom: 4 },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 10,
    fontSize: 16,
  },
});

```

```
export default TextInputPropsDemo;
```

3. KeyboardAvoidingView & Keyboard Management

When an input field is positioned near the lower portion of the screen, launching the software keyboard can obscure the focused field. KeyboardAvoidingView shifts layout dimensions dynamic to keyboard visibility.

Platform Differences

- **iOS:** Native view dimensions remain stationary when soft keyboards appear. The layout requires behavior="padding" or active offsets (keyboardVerticalOffset) to push content up smoothly.
- **Android:** Handles layout adjustments natively via system settings (windowSoftInputMode). Setting behavior="height" or leaving default handling is standard.

TypeScript

```
behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
```

Complete Implementation Example

TypeScript

```
import React, { useRef, useState } from 'react';
import {
  SafeAreaView,
  View,
  Text,
  TextInput,
  TouchableOpacity,
  ScrollView,
  KeyboardAvoidingView,
  TouchableWithoutFeedback,
  Keyboard,
  Platform,
  StyleSheet,
} from 'react-native';
```

```
const AdvancedKeyboardAvoiding = () => {
  const [comments, setComments] = useState([
    { id: 1, text: 'Great post!', author: 'Alice' },
  ];
```

```

    { id: 2, text: 'Thanks for sharing!', author: 'Bob' },
  ]);
  const [newComment, setNewComment] = useState("");

  const addComment = () => {
    if (newComment.trim()) {
      setComments((prev) => [
        ...prev,
        { id: Date.now(), text: newComment, author: 'You' },
      ]);
      setNewComment("");
      Keyboard.dismiss();
    }
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.header}>
        <Text style={styles.headerTitle}>Comments</Text>
      </View>

      <KeyboardAvoidingView
        style={styles.keyboardAvoidingView}
        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
        keyboardVerticalOffset={Platform.OS === 'ios' ? 90 : 0}
      >
        <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
          <View style={styles.contentContainer}>
            <ScrollView style={styles.commentsScrollView}>
              {comments.map((item) => (
                <View key={item.id} style={styles.commentItem}>
                  <Text style={styles.commentAuthor}>{item.author}</Text>
                  <Text style={styles.commentText}>{item.text}</Text>
                </View>
              ))}
            </ScrollView>

            { /* Input Bar */ }
            <View style={styles.inputContainer}>
              <TextInput

```

```

        style={styles.input}
        value={newComment}
        onChangeText={setNewComment}
        placeholder="Write a comment..."
        returnKeyType="send"
        onSubmitEditing={addComment}
      />
      <TouchableOpacity style={styles.sendButton} onPress={addComment}>
        <Text style={styles.sendButtonText}>Send</Text>
      </TouchableOpacity>
    </View>
  </View>
</TouchableWithoutFeedback>
</KeyboardAvoidingView>
</SafeAreaView>
);
};

```

```

const styles = StyleSheet.create({
  safeArea: { flex: 1, backgroundColor: '#f5f5f5' },
  header: { padding: 16, backgroundColor: '#fff', borderBottomWidth: 1,
borderColor: '#eee' },
  headerTitle: { fontSize: 18, fontWeight: 'bold' },
  keyboardAvoidingView: { flex: 1 },
  contentContainer: { flex: 1 },
  commentsScrollView: { flex: 1, padding: 16 },
  commentItem: { backgroundColor: '#fff', padding: 12, borderRadius: 8,
marginBottom: 8 },
  commentAuthor: { fontWeight: 'bold', fontSize: 12, color: '#666' },
  commentText: { fontSize: 14, marginTop: 2 },
  inputContainer: {
    flexDirection: 'row',
    padding: 12,
    backgroundColor: '#fff',
    borderTopWidth: 1,
    borderColor: '#eee',
  },
  input: {
    flex: 1,
    height: 40,

```

```

borderWidth: 1,
borderColor: '#ddd',
borderRadius: 20,
paddingHorizontal: 16,
marginRight: 8,
},
sendButton: {
  backgroundColor: '#007AFF',
  borderRadius: 20,
  paddingHorizontal: 16,
  justifyContent: 'center',
},
sendButtonText: { color: '#fff', fontWeight: 'bold' },
});

```

export default AdvancedKeyboardAvoiding;

4. End-to-End Registration Form with Validation & Ref Navigation

Here is a practical, production-ready profile creation screen featuring **Controlled State Management, Validation Rules, Focus Swapping via Refs, and Keyboard Handling**.

TypeScript

```

import React, { useRef, useState } from 'react';
import {
  ScrollView,
  View,
  Text,
  TextInput,
  Button,
  Switch,
  Alert,
  KeyboardAvoidingView,
  TouchableWithoutFeedback,
  Keyboard,
  Platform,
  StyleSheet,
  ActivityIndicator,
} from 'react-native';

```

```
type FormData = {
  fullName: string;
  email: string;
  age: string;
  password: string;
};
```

```
const ProductionFormExample = () => {
  const [formData, setFormData] = useState<FormData>({
    fullName: "",
    email: "",
    age: "",
    password: "",
  });
```

```
  const [errors, setErrors] = useState<Record<string, string>>({});
  const [agreeToTerms, setAgreeToTerms] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);
```

```
  // Focus Navigation Refs
```

```
  const emailRef = useRef<TextInput>(null);
  const ageRef = useRef<TextInput>(null);
  const passwordRef = useRef<TextInput>(null);
```

```
  const handleChange = (field: keyof FormData, value: string) => {
    setFormData((prev) => ({ ...prev, [field]: value }));
    if (errors[field]) {
      setErrors((prev) => ({ ...prev, [field]: "" }));
    }
  };
};
```

```
  const validate = () => {
    const newErrors: Record<string, string> = {};
```

```
    if (!formData.fullName.trim()) newErrors.fullName = 'Name is required';
    if (!formData.email.trim()) {
      newErrors.email = 'Email is required';
    } else if (!/^[a-zA-Z0-9+@-]+\.[a-zA-Z0-9+@-]+$/i.test(formData.email)) {
      newErrors.email = 'Enter a valid email address';
    }
  };
};
```

```

    }
    if (formData.age && (isNaN(Number(formData.age)) || Number(formData.age)
< 1)) {
      newErrors.age = 'Enter a valid age';
    }
    if (!formData.password || formData.password.length < 6) {
      newErrors.password = 'Password must be at least 6 characters';
    }
    if (!agreeToTerms) {
      newErrors.terms = 'You must accept the terms';
    }

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

```

```

const handleSubmit = async () => {
  if (!validate()) return;

```

```

  setIsSubmitting(true);
  // Simulating API Call
  setTimeout(() => {
    setIsSubmitting(false);
    Alert.alert('Success', 'Profile created successfully!');
  }, 1200);
};

```

```

return (
  <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
    <KeyboardAvoidingView
      style={styles.container}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <ScrollView contentContainerStyle={styles.scrollContent}
keyboardShouldPersistTaps="handled">
        <Text style={styles.title}>Create Account</Text>

        { /* Full Name */ }
        <View style={styles.field}>
          <Text style={styles.label}>Full Name *</Text>

```

```

<TextInput
  style={[styles.input, errors.fullName ? styles.inputError : null]}
  value={formData.fullName}
  onChangeText={(text) => handleChange('fullName', text)}
  placeholder="John Doe"
  returnKeyType="next"
  onSubmitEditing={() => emailRef.current?.focus()}
  blurOnSubmit={false}
/>
{errors.fullName && <Text
style={styles.error}>{errors.fullName}</Text>}
</View>

{/* Email */}
<View style={styles.field}>
  <Text style={styles.label}>Email *</Text>
  <TextInput
    ref={emailRef}
    style={[styles.input, errors.email ? styles.inputError : null]}
    value={formData.email}
    onChangeText={(text) => handleChange('email', text)}
    keyboardType="email-address"
    autoCapitalize="none"
    placeholder="john@example.com"
    returnKeyType="next"
    onSubmitEditing={() => ageRef.current?.focus()}
    blurOnSubmit={false}
  />
  {errors.email && <Text style={styles.error}>{errors.email}</Text>}
</View>

{/* Age */}
<View style={styles.field}>
  <Text style={styles.label}>Age</Text>
  <TextInput
    ref={ageRef}
    style={[styles.input, errors.age ? styles.inputError : null]}
    value={formData.age}
    onChangeText={(text) => handleChange('age', text)}
    keyboardType="numeric"
  />

```

```

        maxLength={3}
        placeholder="25"
        returnKeyType="next"
        onSubmitEditing={() => passwordRef.current?.focus()}
        blurOnSubmit={false}
      />
      {errors.age && <Text style={styles.error}>{errors.age}</Text>}
    </View>

    {/* Password */}
    <View style={styles.field}>
      <Text style={styles.label}>Password *</Text>
      <TextInput
        ref={passwordRef}
        style={[styles.input, errors.password ? styles.inputError : null]}
        value={formData.password}
        onChangeText={(text) => handleChange('password', text)}
        secureTextEntry
        placeholder="••••••••"
        returnKeyType="done"
        onSubmitEditing={handleSubmit}
      />
      {errors.password && <Text
style={styles.error}>{errors.password}</Text>}
    </View>

    {/* Terms Toggle */}
    <View style={styles.switchRow}>
      <Switch value={agreeToTerms} onValueChange={setAgreeToTerms} />
      <Text style={styles.switchLabel}>I agree to the Terms &
Conditions</Text>
    </View>
    {errors.terms && <Text style={styles.error}>{errors.terms}</Text>}

    {/* Submit Action */}
    <View style={styles.submitContainer}>
      {isSubmitting ? (
        <ActivityIndicator color="#007AFF" />
      ) : (
        <Button title="Register" onPress={handleSubmit} color="#007AFF" />
      )}
    </View>
  </Form>
</View>

```

```

    })
  </View>
</ScrollView>
</KeyboardAvoidingView>
</TouchableWithoutFeedback>
);
};

```

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  scrollContent: { padding: 20, justifyContent: 'center' },
  title: { fontSize: 24, fontWeight: 'bold', marginBottom: 20, textAlign: 'center' },
  field: { marginBottom: 14 },
  label: { fontSize: 14, fontWeight: '600', marginBottom: 4, color: '#333' },
  input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8, padding: 12,
fontSize: 16 },
  inputError: { borderColor: '#d9534f' },
  error: { color: '#d9534f', fontSize: 12, marginTop: 2 },
  switchRow: { flexDirection: 'row', alignItems: 'center', marginVertical: 10 },
  switchLabel: { marginLeft: 10, fontSize: 14 },
  submitContainer: { marginTop: 16 },
});

```

```

export default ProductionFormExample;

```